API Documentation

REST endpoints, contracts, authentication, and testing reference

Project	HAL Smart PDF Approval & QR Verification System
Purpose	A practical API reference for development, debugging, and interview explanation.
Stack	Spring Boot 3.5.16 · Java 21 · React/Vite · PostgreSQL · JWT · PDFBox · ZXing · Docker

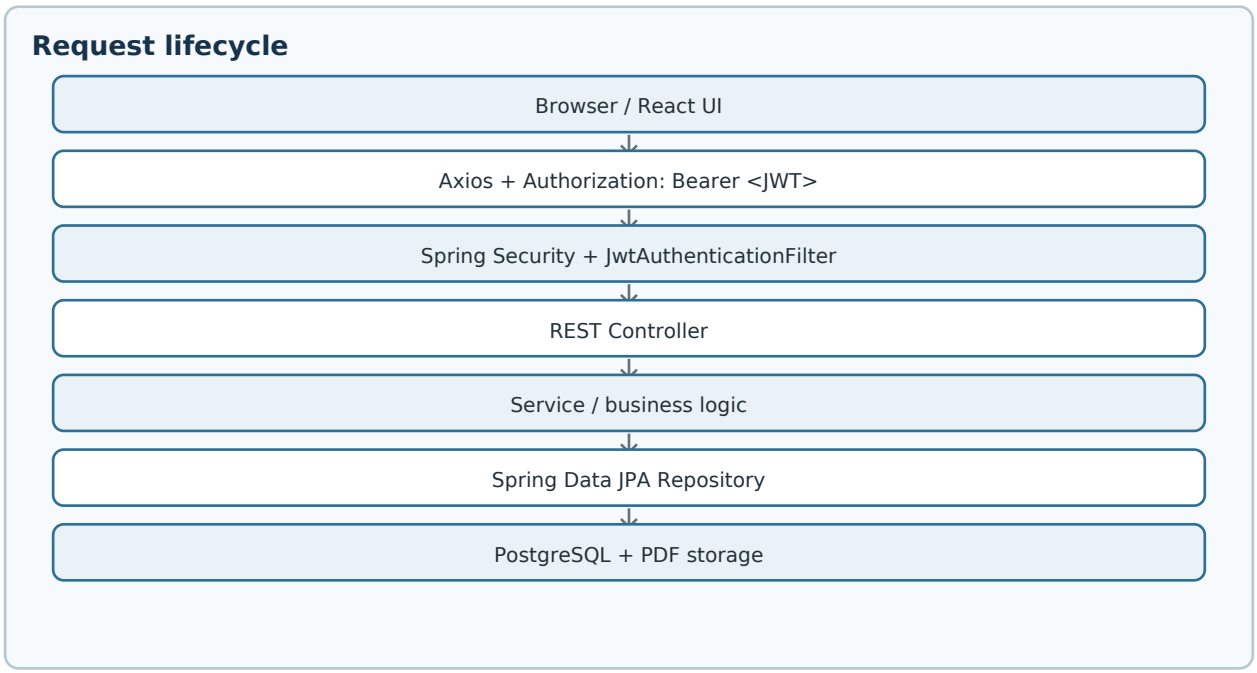
Prepared as project documentation and presentation-ready reference material.

Contents

A structured reference for development, documentation, and interviews.

1. API Overview

The React/Vite frontend communicates with a Spring Boot REST API through Axios. Login is public; application operations are protected by JWT authentication.



Conventions

- JSON is used for normal requests/responses.
- PDF upload uses multipart/form-data.
- Authenticated requests send an Authorization Bearer token.
- Document workflow states include PENDING, APPROVED, and REJECTED.

2. Authentication

POST /auth/login

```
{
"email": "admin@hal.com",
"password": "Admin@123"
}
```

The server locates the employee, verifies the BCrypt password, creates a signed JWT, and returns login information to the frontend.

Authentication sequence

- Frontend sends credentials.
- AuthService retrieves Employee by email.
- BCryptPasswordEncoder verifies the password hash.
- JwtService signs the token using the configured secret and expiration.
- Frontend uses the token for protected requests.
- JwtAuthenticationFilter validates the token and establishes authentication.

3. Document APIs

POST /documents/upload

Consumes multipart/form-data containing the PDF and employee identifier. A successful upload creates a PENDING document record.

```
Example response:
{
"id": 3,
"documentName": "test-upload.pdf",
"status": "PENDING",
"uploadedBy": "Test Employee"
}
```

PATCH /documents/{id}/approve

Authenticated administrator operation. Approval metadata such as managerId and comments is supplied to the backend; the service updates the document and invokes QR/PDF processing.

PATCH /documents/{id}/reject

Authenticated review operation that records the rejection decision and comments according to the service implementation.

GET /documents/{id}/download

Returns the approved/QR-enhanced PDF for an authorized user.

4. Contracts & File Handling

Approval request

```
{
"managerId": 1,
"comments": "Approved for use"
}
```

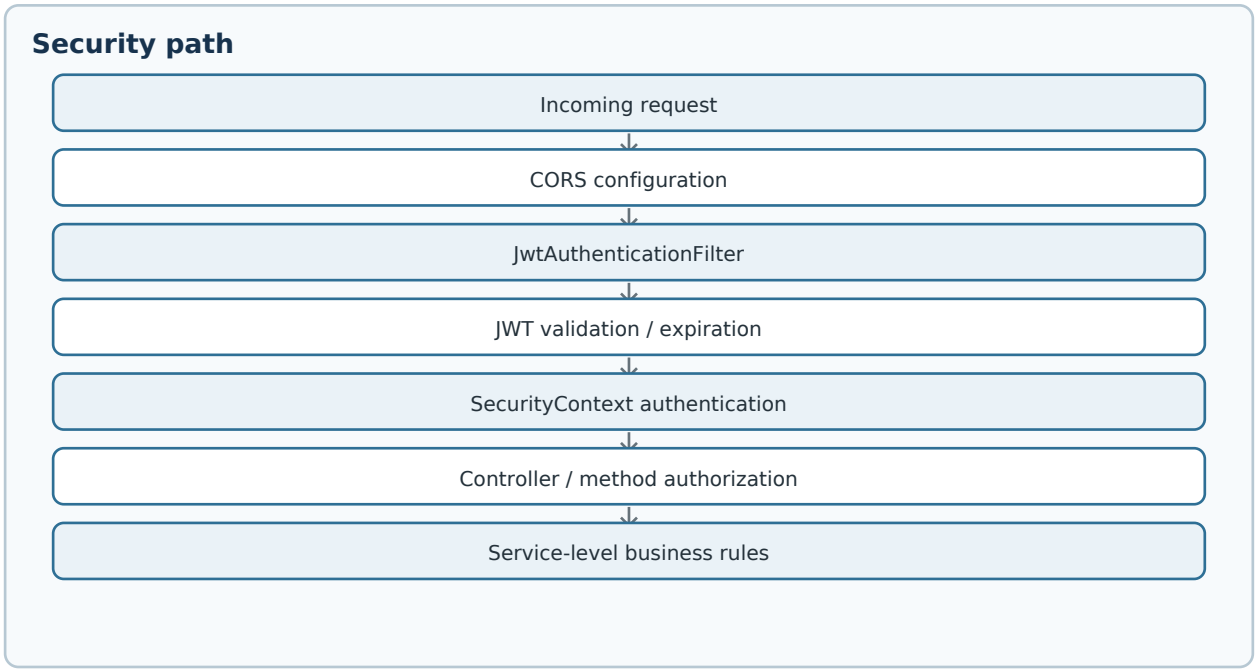
Typical document response

```
{
"id": 3,
"documentName": "example.pdf",
"status": "APPROVED",
"uploadedBy": "Test Employee",
"createdAt": "...",
"approvedAt": "...",
"comments": "Approved for use"
}
```

Processing model

- Database stores document metadata and file path information.
- PDF binary content is processed by the backend.
- ZXing generates the QR image.
- PDFBox modifies the approved PDF and the resulting output is exposed through download.

5. Security



Public endpoints

/ and /auth/login are public. The project also exposes a QR demonstration endpoint used for development/testing.

Protected endpoints

Other application operations require authentication. Method security is enabled so role/authority checks can be applied where needed.

Production rule

FRONTEND_URL must match the deployed frontend origin, while JWT_SECRET must be non-empty and sufficiently strong.

6. Error Handling

Symptom	Likely cause	First check
400 login	Validation/business issue	Backend logs + employee record
401	Missing/invalid/expired JWT	Authorization header + token
403	Role/authority restriction	JWT role + method security
500 database	Schema/configuration	Datasource + Hibernate logs
CORS browser error	Origin not allowed	FRONTEND_URL + Network tab
PDF download error	File/output path issue	Server filesystem + document row

7. Testing Checklist

- Verify backend health endpoint.
- Login as employee and admin.
- Upload a sample PDF.
- Confirm PENDING state.
- Approve/reject as administrator.
- Confirm APPROVED output and QR.
- Download and inspect the generated PDF.
- Verify protected requests fail without JWT.
- Verify deployed frontend passes CORS checks.

Interview-ready explanation

“My frontend communicates with a Spring Boot REST API. Login verifies a BCrypt password and returns a JWT. Protected requests carry the JWT in the Authorization header, and a custom filter validates it before controller execution. Document APIs handle upload, review, approval/rejection, and download, while JPA repositories persist the metadata in PostgreSQL.”

End of API Documentation